Suggested Teaching Guidelines for

Web Programming Technologies

PGCP-AC, August 2026

Duration: 150 Hours: 60 Theory (T) + 60 Lab (L) + 30 Self Learning (SL)

Objective: To equip students with the skills to design, develop, and deploy responsive, interactive, and modern web applications

Prerequisites: Programming fundamentals and familiarity with using web browsers and development tools.

Evaluation method: Theory exam – 40%, Lab exam – 40%, Internal Assessment – 20%

List of Course Material

Text Book:
No specific courseware for the module, faculty may share some course materials.

Reference book:

1. Web Development with Node and Express, 2014 edition, Ethan Brown / O'Reilly
2. JavaScript: The Definitive Guide, 7th Edition, David Flanagan / O'Reilly
3. React in Depth, 2024 edition, Morten Barklund / O'Reilly

Session 1: Architecture of Web (2T)

- History and Working of the Internet
- Internet Protocols
- Domain Names; Domain Name Service Servers
- HTTP Protocols
  - Methods – GET, POST, HEAD, PUT, DELETE, etc.
  - Status codes
  - Stateless nature of the protocol and HTTP Session
  - HTTPS
- Architecture of the Web, Web servers – IIS, Apache server

Session 2: HTML (2T + 2L)

- Introduction to HTML5
- Basic HTML Tags
  - Alignment
  - Headings
  - Anchor
  - Paragraph
  - Image
  - Lists
  - Tables
  - iFrames

Suggested Teaching Guidelines for

Web Programming Technologies

PGCP-AC, August 2026

- HTML5
  - New elements
  - Attributes
  - ARIA accessibility
  - Canvas
  - HTML5 Validation
  - Audio & Video Support
  - Geo-location Support
- HTML Forms & Controls
  - Input, Text Area, Radio Button, Checkbox, Dropdown, Submit, Reset, Button, etc.
- Introduction to Document Object Model(DOM)

Lab:

- Create a HTML form for building your resume.

Session 3: Cascading Style Sheets (2T + 2L)

- Introduction to CSS
- Inline CSS, Internal CSS, External CSS
- CSS Fonts
- CSS Box Model
- id and class Attributes
- HTML Style Tags
- Linking style sheets

Lab:

- Apply inline, internal and external CSS to style the resume

Session 4: Responsive Web Design – Tailwind CSS (2T + 2L + 2SL)

- Introduction to utility-first CSS
- Tailwind setup and configuration
- Core utilities: spacing, typography, colours, borders
- Responsive prefixes
- Dark mode with Tailwind; custom themes; @apply for component extraction
- Responsive Design with Tailwind CSS

Lab:

- Rebuild résumé form using Tailwind CSS utility classes
- Make the layout fully responsive using Tailwind breakpoints

Suggested Teaching Guidelines for

Web Programming Technologies

PGCP-AC, August 2026

Sessions 5 & 6: JavaScript Fundamentals (4T + 4L + 2SL)

- Introduction to JavaScript
- Variables (var, let, const)
- Statements, Operators, Expressions
- Control Structures
- Strings, String Methods
- Numbers, Number Methods
- Boolean Values
- Dates, Date Formats, Date Methods
- Arrays, Array Methods

Lab:

- Practice writing basic JavaScript programs for better understanding of the language constructs

Sessions 7 & 8: JavaScript OOP (4T + 4L + 2SL)

- Objects: Definitions, Properties, Methods, Prototypes
- Functions: Definitions, Parameters, Invocation, Closures
- Introduction to Object Oriented Programming in JS
  - Methods
  - Constructor
  - Inheritance
  - Encapsulation
  - Abstraction
  - Polymorphism

Lab:

- Sort a list of elements by implementing a sorting algorithm.
- Write a JavaScript program to list the properties of a JavaScript object.

Sessions 9, 10 & 11: DOM, Forms, Regex and Debugging (5T + 6L + 2SL)

- Document Object Model (DOM)
  - Object hierarchy in JavaScript
  - Elements
  - Events
  - Methods
  - Manipulation
- Forms, Forms API, Forms Validation
- Regular Expressions
- Errors, Debugging
- Introduction to Browser Dev Tool

- Code quality via ESLint
- JSON: JavaScript Object Notation (JSON)
  - Introduction and need of JSON
  - JSON Syntax Rules
  - JSON Objects
  - JSON Arrays
  - JSON Files
  - JSON parsing

Lab:

- Write a JavaScript function to get First and Last name from the previously created Resume form
- Validate the entire Resume form using client-side JavaScript
- Write a JavaScript function to validate whether a given value is RegEx or not.
- Fetch data from a JSON endpoint using Fetch API and display results with error handling
- Create a JSON object, array, and file to store a cricket match (or any team sport) scoreboard.

Sessions 12 & 13: TypeScript (4T + 4L + 2SL)

- TypeScript Features
- TypeScript Setup
- Primitive types, type inference, union types, type aliases, literal types
- Interfaces vs type aliases; optional and readonly properties; enums; tuples
- Generics — T, K, V patterns; type narrowing: typeof, instanceof, discriminated unions
- TypeScript with the DOM: typed event handlers, HTMLElement generics
- TypeScript with Axios: typed API responses; async/await with explicit return types
- Using TypeScript with React: typing props, state, event handlers, refs

Lab:

- Setup a TypeScript project and convert a plain JS file to TS and fix all type errors
- Define interfaces for an API response, type a Fetch/Axios call end-to-end
- Build a typed DOM utility module, write a generic function used in 2+ contexts
- Convert the resume from JS to TS

Session 14: AJAX and Axios HTTP Client (2T + 2L + 2SL)

- Introduction to AJAX and Axios
  - Web Services and AJAX
  - The Axios instance and its config
  - Handling request and response
  - Error Handling

Lab:

- Live news headlines using Axios with typed TypeScript API response

Session 15: Introduction to Node.js (2T + 2L)

- Introduction to Node.js
- Browser JS vs. Node.js
- Node.js REPL
- ECMAScript 2015 (ES6+)

Lab:

- Install Node.js 22 LTS version and verify with node -v and npm -v
- Write a recursive function in Node.js
- Write a Node program that prints all the numbers between 1 and 100, each on a separate line. A few caveats:
  - if the number is divisible by 3, print "foo"
  - if the number is divisible by 5, print "bar"
  - if the number is divisible by both 3 and 5, print "foobar"

Sessions 16, 17 & 18: Node.js Asynchronous Programming (6T + 6L + 4SL)

- Introduction to Asynchronous programming and callbacks
- Promises and async& await
- The Event Loop and Timers
- Async error handling patterns
  - try/catch with async/await
  - Promise.all
  - Promise.allSettled
- Node.js streams and piping: practical async I/O patterns

Lab:

- Assignment on JavaScript callback functions
- Assignment on Timers, Promises, and Async& Await
- Implement Promise.all to fetch multiple endpoints concurrently and handle partial failures

Session 19: Node.js Modules (3T + 3L)

- Understanding Node modules, exports, and require
- Introduction to npm
  - package.json and package-lock.json files
  - Install, update, and manage package dependencies
  - Local and global packages

- File I/O – Sync & Async Methods
- HTTP Module – Building an HTTP server
- Developing a Node web application

Lab:

- Create a module and import it in other programs
- Install a module/package using npm
- Write a program to create a new file and write some content to it in synchronous mode and read and display file contents on standard output in async mode
- Build a simple Node.js web application serving both HTTP GET and POST methods

Sessions 20 & 21: Introduction to Express (4T + 5L + 2SL)

- Introduction to Express
- Application, Request and Response Objects
- Routes and Middlewares
- Templates, Template Engines, and Rendering Views

Lab:

- Use Node and Express to write a simple web application that consists of at least 5 route implementations
- Rebuild any previous Node assignment using Express and a template engine

Session 22: Introduction to React (2T + 2L + 2SL)

- Introduction to React
- React Elements and Components
- Function Components
- Working with Props
  - Compose components
  - Render components
  - Declutter components

Lab:

- Rebuild any previous plain HTML lab assignment using React functional components
- Build a React Clock app showing time (hh:mm:ss) of any three countries

Sessions 23, 24 & 25: React Hooks (6T + 6L + 4SL)

- Introduction to React Hooks
  - useState
  - useEffect

  - useContext
  - useMemo
  - useCallabck
  - useSelector
- Props vs. State vs. Context
- Handling events
- Conditional rendering

Lab:

- Update Clock app time using useState and useEffect
- Implement the following items in the React Clock app
  - Update the time (hh:mm:ss) using State and Lifecycle methods
  - Add a close functionon each rendered clock component
  - Assign background color of rendered clock components based on AM, PM

Sessions 26, 27 & 28: React (6T + 6L + 4SL)

- Lists and Keys
- Rendering Multiple Components
- Basic List Component
- Working with forms and inputs
- useRef, useCallback and useMemo Hooks
- Refs and the DOM
- Lifting state up

Lab:

- Implement and integrate a new feature in the React Clock app where one can select a country time zone from dropdown list and click on “Add” button to render it
- Use useRef to focus an input field programmatically
- Extract clock logic into a custom useClockTick hook
- Demonstrate useCallback preventing unnecessary child re-renders

Sessions 29 & 30: React (4T + 4L + 2SL)

- Error Boundaries
- Composition vs. Inheritance
  - Containment
  - Specialization
- React Router: Client-side routing, SPA Navigation
- Routes, Route, Link, NavLink, useNavigate, useParams
- Nested routes and layout routes
- Thinking in React

Suggested Teaching Guidelines for

Web Programming Technologies

PGCP-AC, August 2026

Lab:

- Implement error boundaries at appropriate places in the React Clock app
- Add React router to the clock app
    - Implement useNavigate for programmatic navigation and useParams for dynamic routes